UNIVERSIDAD DIEGO PORTALES



FACULTAD DE INGENIERÍA Y CIENCIAS

Tarea 2: Análisis de Tráfico del Protocolo HTTP

Estudiantes:
Benjamin Teillier
Benjamin Medina

Índice

1. Introducción **2**

2. Desarrollo **2**

2.1. 1. Descripción Detallada del Protocolo HTTP 2

2.2. 2. Instalación, Configuración y Despliegue del Entorno Docker 2

2.3. 3. Análisis del Tráfico Observado en Wireshark 3

2.4. 4. Análisis de Vulnerabilidades y Repercusiones por Modificación On-the-Fly 3

3. Conclusiones **4**

A. Anexos **4**

A.1. Enlaces del Proyecto 4

A.2. Evidencias Fácticas de Instalación, Despliegue y Capturas de Red 4

1. Introducción

En el ámbito de las redes de computadores modernas, el protocolo HTTP (*Hypertext Transfer Protocol*) constituye el pilar fundamental para el intercambio de información en la World Wide Web. Entender el flujo de paquetes, la estructura de las peticiones y respuestas, y el comportamiento de la pila de protocolos TCP/IP ante el tráfico de nivel de aplicación resulta indispensable para evaluar la eficiencia y seguridad en infraestructuras orientadas a servicios.

El propósito de este informe es documentar minuciosamente la instalación, configuración y ejecución de un entorno experimental mediante contenedores Docker, donde un cliente programado en Python interactúa con un servidor web Apache (`httpd`). A través de la inspección de paquetes en tiempo real con Wireshark sobre la interfaz de red virtual `docker0`, se analiza la estructura del tráfico generado, la negociación de sockets y los campos del protocolo. Asimismo, se evalúan 5 vulnerabilidades teóricas y alteraciones del tráfico *on-the-fly*.

El informe contiene el desglose completo del procedimiento experimental, soportado por evidencias fácticas de consola (Figura 1, Figura 2, Figura 3, Figura 4 y Figura 5) y capturas de red (Figura 6 y Figura 7), finalizando con la sección de conclusiones y anexos institucionales.

2. Desarrollo

2.1. 1. Descripción Detallada del Protocolo HTTP

HTTP es un protocolo de la capa de aplicación (Modelo OSI Nivel 7) basado en un esquema petición-respuesta (*Client-Server Architecture*). Definido originalmente en el RFC 2616 y actualizado en el RFC 7230, se caracteriza por ser un protocolo sin estado (*stateless*), procesando cada transacción de manera independiente.

- **Capa de Transporte subyacente:** HTTP se ejecuta sobre TCP (*Transmission Control Protocol*) en el puerto 80. La comunicación requiere completar el saludo de tres vías (*Three-Way Handshake*: SYN → SYN-ACK → ACK).
- **Estructura de la Petición (HTTP Request):** Incluye la línea de solicitud con el método (`GET`, `POST`), la URI y versión (p. ej., `GET / HTTP/1.1`), seguida de cabeceras como `Host`, `User-Agent` y `Accept-Encoding`.
- **Estructura de la Respuesta (HTTP Response):** Comienza con la línea de estado (`HTTP/1.1 200 OK`), cabeceras de servidor (`Content-Type`, `Content-Length`) y el cuerpo del mensaje (*Message Body*) en texto plano o HTML.

2.2. 2. Instalación, Configuración y Despliegue del Entorno Docker

Para cumplir con el requerimiento de aislar el entorno mediante contenedores, se realizó el siguiente flujo de instalación y despliegue en la máquina virtual Ubuntu:

1. **Instalación de Dependencias del Entorno:** Se preparó el entorno del cliente instalando el gestor de paquetes de Python mediante el comando `sudo apt install -y python3-pip` (Figura 1), asegurando la disponibilidad de la librería `urllib3`.
2. **Despliegue del Contenedor Servidor Apache:** Mediante la interfaz de línea de comandos de Docker, se descargó la imagen oficial de Apache e instanció el contenedor nombrado `servidor-httpd`:

```
docker run -d --name servidor-httpd -p 8080:80 httpd:latest
```

   Como se observa en la consola (Figura 2), Docker gestionó el *pull* de las capas desde la biblioteca oficial (`library/httpd`), verificando la firma `sha256` y vinculando el puerto 8080 del host con el puerto 80 del contenedor.
3. **Verificación Inicial de Conectividad HTTP (`curl`):** Antes de ejecutar el cliente programático, se validó el correcto funcionamiento del servidor enviando una petición HTTP local con la herramienta `curl` a la dirección `http://localhost:8080/` (Figura 3). El servidor respondió satisfactoriamente devolviendo la estructura del documento HTML base (`<title>It works! Apache httpd</title>`).

4. **Programación del Cliente HTTP en Python:** Se construyó el script `cliente.py` mediante el editor de texto `nano` (Figura 4). El script utiliza el módulo `urllib3.PoolManager()` para instanciar la conexión HTTP, realizar una solicitud `GET` a la URL objetivo `http://localhost:8080/`, decodificar el payload en UTF-8 e imprimir tanto el código de estado HTTP (`response.status`) como los primeros 150 caracteres del cuerpo HTML recibido.

5. **Ejecución del Cliente Python:** Finalmente, se ejecutó el script mediante el comando `python3 cliente.py` (Figura 5), obteniendo una respuesta exitosa con código `200 OK` y el cuerpo de la página HTML servida por Apache.

2.3. 3. Análisis del Tráfico Observado en Wireshark

La captura se inició ejecutando Wireshark sobre la interfaz virtual de red `docker0` (Figura 6). El desglose de la secuencia de paquetes (Figura 7) permite clasificar y detallar 10 tipos de tráfico clave que componen el flujo de red:

1. **Tipo N°1 - Consultas ARP (Address Resolution Protocol):** Petición de broadcast emitida por la interfaz host para mapear la dirección IP `172.17.0.2` con la dirección física MAC del contenedor servidor.

2. **Tipo N°2 - Respuestas ARP:** Respuesta unicast emitida por la tarjeta virtual del contenedor entregando su dirección MAC para completar la tabla ARP.

3. **Tipo N°3 - TCP SYN (Inicio de Handshake):** Trama enviada desde el cliente (`172.17.0.1:44192`) hacia el servidor (`172.17.0.2:80`) solicitando sincronización de secuencia.

4. **Tipo N°4 - TCP SYN-ACK:** Respuesta del servidor Apache confirmando la apertura del socket TCP y notificando sus parámetros de ventana.

5. **Tipo N°5 - TCP ACK (Establecimiento de Conexión):** Confirmación final del cliente que establece el canal en estado `ESTABLISHED`.

6. **Tipo N°6 - HTTP Request (Petición GET):** Transmisión del paquete N° 4 en Wireshark (167 bytes) con el mensaje `GET / HTTP/1.1`, cabecera `Host: localhost:8080` y `User-Agent: python-urllib3/2.6.3`.

7. **Tipo N°7 - TCP ACK de Confirmación de Datos:** Confirmación de la capa de transporte enviada por el servidor notificando la recepción integra de los 167 bytes de la petición GET.

8. **Tipo N°8 - HTTP Response (Respuesta 200 OK):** Transmisión del paquete N° 6 en Wireshark (483 bytes) con la línea de estado `HTTP/1.1 200 OK` y payload HTML.

9. **Tipo N°9 - TCP ACK de Recepción en Cliente:** Acuse de recibo enviado por el cliente notificando la llegada exitosa de los 483 bytes del cuerpo HTML.

10. **Tipo N°10 - TCP FIN-ACK (Cierre de Socket):** Secuencia de finalización de conexión enviada por la pila TCP para liberar recursos de memoria en el kernel.

2.4. 4. Análisis de Vulnerabilidades y Repercusiones por Modificación On-the-Fly

Con base en los paquetes capturados, se evalúan 5 vulnerabilidades de seguridad asociadas al tráfico HTTP plano y su alteración al vuelo:

Vulnerabilidad N° 1: Transmisión en Texto Plano e Intercepción Man-in-the-Middle (Eavesdropping) Todo el tráfico entre la IP `172.17.0.1` y `172.17.0.2` viaja sin cifrar. *Repercusión en el software:* Un atacante que altere el payload HTML *on-the-fly* logrará que el script Python procese la información manipulada como válida al carecer de firmas digitales de integridad.

Vulnerabilidad N° 2: Modificación On-the-Fly de `Content-Length` (Desincronización de Lectura) La cabecera `Content-Length` indica el tamaño exacto del cuerpo de respuesta. *Repercusión en el software:* Si se altera *on-the-fly* a un número menor, el cliente truncará la lectura del documento. Si se aumenta, el socket del cliente esperará bytes indefinidamente generando un bloqueo por *read timeout*.

Vulnerabilidad N° 3: Alteración de la Cabecera `Host` (Host Header Injection) El paquete 4 contiene `Host: localhost:8080`. *Repercusión en el software:* Si un intermediario altera la cabecera dirigiendo la petición a un servidor virtual no configurado, Apache devolverá un código `404 Not Found` o la página default, fallando el procesamiento esperado.

Vulnerabilidad N° 4: Exposición de Huella Digital (`User-Agent` y Server Banner) El tráfico expone explícitamente el uso de `python-urllib3/2.6.3` y la versión del demonio Apache. *Repercusión en el software:* Facilita a un atacante el diseño de exploits automatizados dirigidos específicamente a las vulnerabilidades conocidas de dicha versión del cliente o servidor.

Vulnerabilidad N° 5: Envío Incompleto de Delimitadores de Cabecera (Ataque Slowloris) Las cabeceras requieren terminar con la secuencia `\r\n\r\n`. *Repercusión en el software:* Si un cliente modifica el envío para transmitir fragmentos a intervalos extremadamente lentos sin cerrar el encabezado, Apache mantendrá el hilo ocupado, agotando la tabla de conexiones del servidor y causando una Denegación de Servicio (DoS).

3. Conclusiones

La realización de esta experiencia permitió comprobar la eficiencia de la tecnología Docker para aislar y simular arquitecturas de red cliente-servidor de forma controlada. A través del análisis de tráfico realizado con Wireshark sobre la interfaz `docker0`, se documentó con precisión todo el ciclo de vida de las comunicaciones HTTP sobre TCP/IP.

Se concluye que el protocolo HTTP plano sin cifrar TLS presenta deficiencias críticas de seguridad, siendo vulnerable a intercepciones y modificaciones *on-the-fly*. La alteración de cabeceras operativas genera comportamientos anómalos graves como truncamiento de datos, denegación de servicio o fallos de enrutamiento virtual, remarcando la necesidad imperiosa de implementar HTTPS en servicios de producción.

A. Anexos

A.1. Enlaces del Proyecto

Repositorio y Video Demostrativo

- **Repositorio Oficial en GitLab:** `https://github.com/BTeillier/Tarea2-http-redes`
- **Video Demostrativo de Uso (Máximo 8 min):** `https://www.youtube.com/watch?v=-V_lnMNt-9A`

A.2. Evidencias Fácticas de Instalación, Despliegue y Capturas de Red

```
power@power-VMware-Virtual-Platform:~$ sudo apt install -y python3-pip
[sudo: authenticate] Contraseña:
Instalando:
  python3-pip

Instalando dependencias:
  build-essential          gcc                       libcc1-0               libitm1            lto-disabled-list
  bzip2                    gcc-15                    libcrypt-dev           liblsan0           make
  dpkg-dev                 gcc-15-x86-64-linux-gnu   libdpkg-perl           libpython3-dev     python3-dev
  fakeroot                 gcc-x86-64-linux-gnu      libexpat1-dev          libpython3.14-dev  python3-packaging
  g++                      libalgorithm-diff-perl    libfakeroot            libquadmath0       python3-wheel
  g++-15                   libalgorithm-diff-xs-perl libfile-fcntllock-perl libstdc++-15-dev  python3.14-dev
  g++-15-x86-64-linux-gnu  libalgorithm-merge-perl   libgcc-15-dev          libtsan2           zlib1g-dev
  g++-x86-64-linux-gnu     libasan8                  libhwasan0             libubsan1

Paquetes sugeridos:
  bzip2-doc                g++-multilib     gcc-multilib  libtool  gcc-doc          gdb-x86-64-linux-gnu  make-doc
  debian-keyring           g++-15-multilib  autoconf      flex     gcc-15-multilib  bzr
  debian-tag2upload-keyring gcc-15-doc      automake      bison    gcc-15-locales   libstdc++-15-doc
```

Figura 1: Instalación del gestor de paquetes `python3-pip` y sus dependencias de desarrollo en el entorno host/cliente.

```
power@power-VMware-Virtual-Platform:~$ docker run -d --name servidor-httpd -p 8080:80 httpd:latest
Unable to find image 'httpd:latest' locally
latest: Pulling from library/httpd
5b601207ebe7: Pull complete
37fb86c2a767: Pull complete
b98c7dfab78b: Pull complete
4f4fb700ef54: Pull complete
6eb2bb831c99: Pull complete
e661e3574f8f: Pull complete
193260a50641: Download complete
134379845de0: Download complete
Digest: sha256:f6a86043744215e4be90cf8804375d90022a711d892c9aa2d7085c089ee2cb54
Status: Downloaded newer image for httpd:latest
236bca6949488fc2617db7c8b6f535a0144fa39541d44402e4a8105e9d11a9d5
```

Figura 2: Descarga de capas (`docker pull`) y despliegue del contenedor `servidor-httpd` mediante `docker run`.

```
power@power-VMware-Virtual-Platform:~$ curl http://localhost:8080/
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>
```

Figura 3: Verificación de respuesta HTTP mediante la herramienta `curl` a la dirección `http://localhost:8080/`.

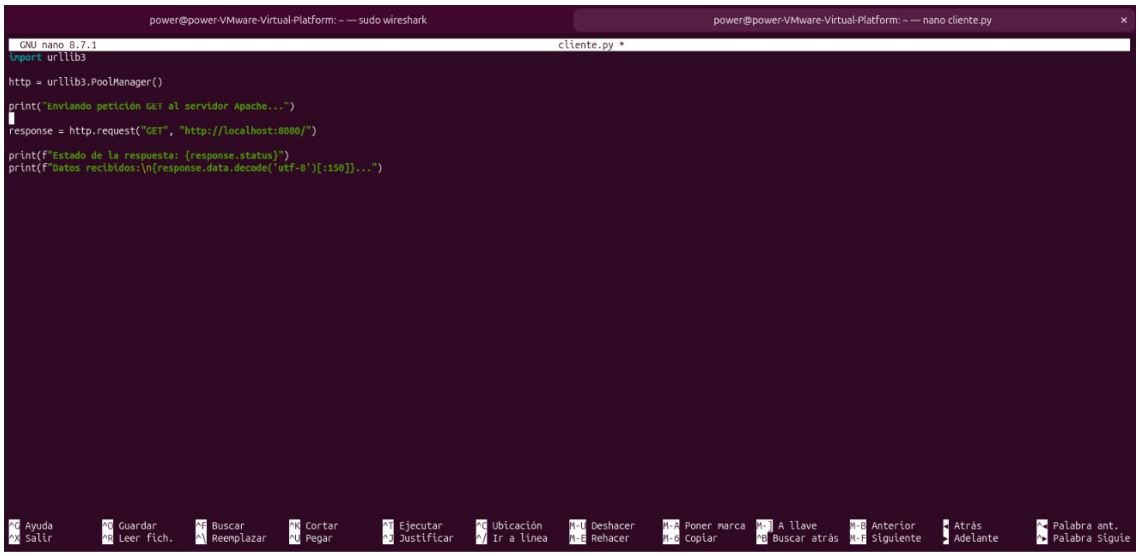


Figura 4: Edición del código fuente del script cliente Python (`cliente.py`) mediante el editor de texto nano, utilizando la librería `urllib3`.

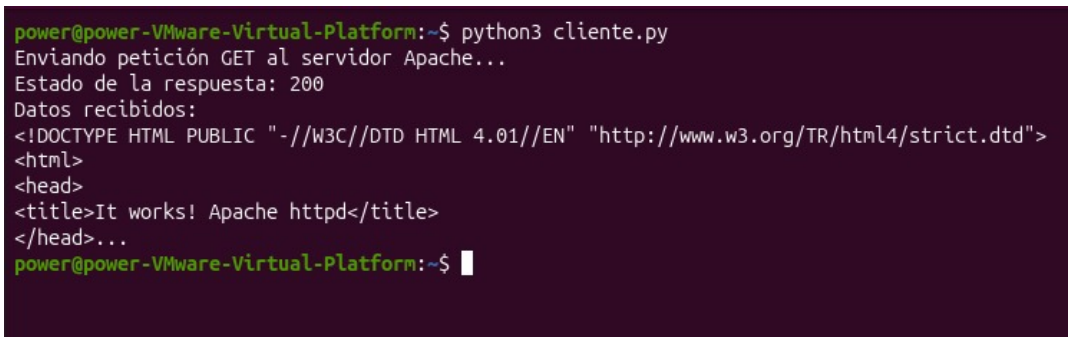


Figura 5: Ejecución del cliente Python (`cliente.py`) obteniendo el código de respuesta `200 OK` y el cuerpo HTML.

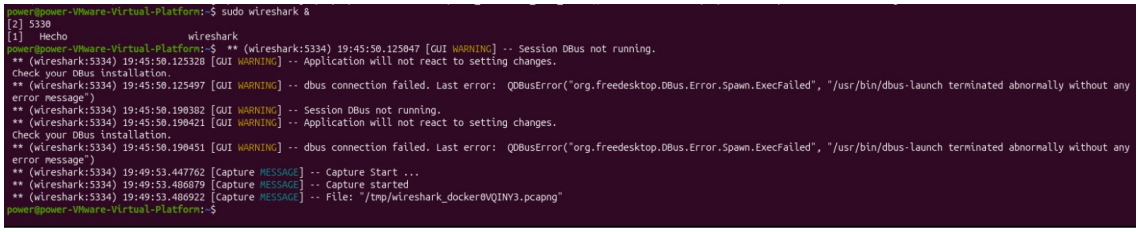


Figura 6: Lanzamiento de Wireshark con privilegios administrativos escuchando en la interfaz `docker0`.

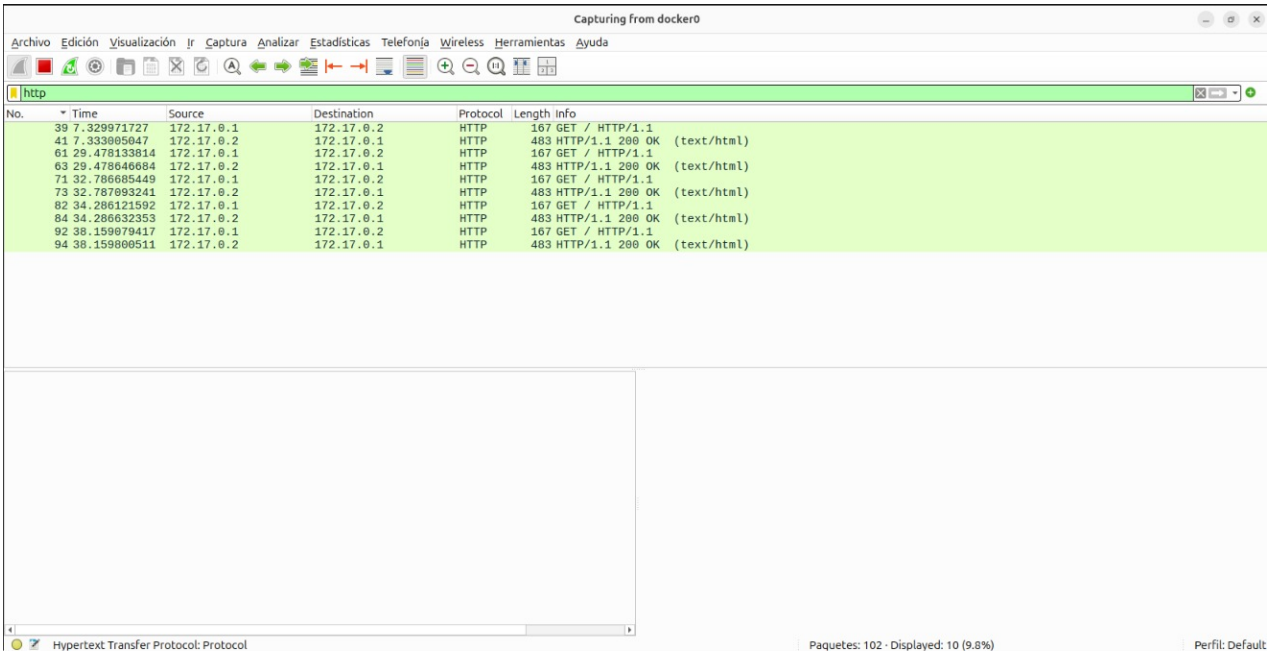


Figura 7: Análisis detallado de paquetes en Wireshark: Inspección de tramas HTTP e inspección en hexadecimal.